Power-Demand-Forecast - 完全実装仕様書（GitHub Pages版）

作成日時: 2026年1月15日
ステータス: Production Ready (GitHub Actions稼働中)
公開URL: https://13236584.github.io/Power-Demand-Forecast/
GitHub リポジトリ: https://github.com/13236584/Power-Demand-Forecast
ポイント: 0円運用
作成者: 13236584
作成日: 2026年1月15日

✅ このドキュメントについて

この仕様は、電力需要予測システムの完全な実装仕様書です。GitHub Actions による自動化、Open-Meteo APIの使用、静的サイトによる手間ゼロ運用を含みます。

システム概要

ゴールとビジョン

電力需要を予測し、ピーク時（17:00-21:00）ボイラー稼動あたりコストを最小化する。

- **データ収集**: Open-Meteo API から気象データを取得
- **自動実行**: 毎日実行（JST 6:30）で予測を生成
- **可視化**: Chart.js によるインタラクティブなグラフ
- **完全無料**: GitHub Pages ホスティングで永続的に公開

ワークフロー

```
毎日 JST 6:30 (GitHub Actions trigger)
↓
Open-Meteo API から気象データを取得 (temp_min)
↓
データを保存 (data.js)
↓
特徴量生成 (lagged_temp, holiday_flag, day_type)
↓
LightGBM 推論 (各ゾーンの Tmax を予測)
↓
metrics.json 生成 → グラフ出力
↓
GitHub Pages 自動デプロイ
↓
閲覧ページ: ゾーン A & B でスコア&コスト比較可能
```

データモデル

特徴量 (X)

```
{
  "date": "2026-01-15",
  "temp_min": -2.5,      // 最低気温 (°C)
  "day_of_week": 4,      // 0=月曜, 6=日曜
  "holiday_flag": 0,     // 0=平日, 1=祝日
  "day_type": "weekday"  // "weekday" | "weekend"
}
```

目的変数 (Y)

```
{
  "date": "2026-01-15",
  "tmax": 12.0           // 最高気温 (°C)
}
```

データソース

データ種別	期間	ソース	フォーマット
電力需要	2016-2025	中部電力	data/raw/中部.csv
気温（過去）	2016-2025	中部電力	data/raw/中部.csv
気温（予測）	過去7日〜今日	Open-Meteo API	data/raw/temp.csv

モデル仕様

1. LightGBM

場所: scripts/train.py

スペックのコード

```
{
  "n_estimators": 100,
  "learning_rate": 0.1,
  "max_depth": -1,
  "num_leaves": 31,
  "random_state": 42
}
```

学習時間: 約3秒
予測時間: < 1秒
精度目標: MAE < 1.0

2. Ensemble (アンサンブル)

アンサンブルのコード

```
{
  "ensemble": [
    {model:lgb, activation:"sig", input_mode:0},
    {model:lgb, activation:"sig", input_mode:1},
    {model:lgb, activation:"sig", input_mode:2}
  ]
}
```

スペックのコード

```
{
  "n_estimators": 100,
  "learning_rate": 0.1,
  "max_depth": -1,
  "num_leaves": 31,
  "random_state": 42,
  "n_jobs": -1  # 全コア使用
}
```

学習時間: 約10秒
予測時間: < 1秒
精度目標: MAE < 1.0

3. BaselineNaive

場所: src/models.py

スペックのコード

```
{
  "n_estimators": 100,
  "learning_rate": 0.1,
  "max_depth": -1,
  "num_leaves": 31,
  "random_state": 42,
  "n_jobs": -1  # 全コア使用
}
```

学習時間: 約10秒
予測時間: < 1秒
精度目標: MAE < 1.5

4. PyCaret (AutoML)

場所: scripts/train_pycaret.py

スペックのコード

```
{
  "session_id": 42,
  "verbose": 0,
  "n_jobs": -1  # 全コア使用
}
```

学習時間: 約60秒
予測時間: < 1秒
精度目標: MAE < 0.8

精度指標

MAPE (Mean Absolute Percentage Error)

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100$$

単位: %
目標値: < 10%

MAE (Mean Absolute Error)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

単位: °C
目標値: < 0.8

GitHub Actions ワークフロー

トリガー:

- Cron 毎日 UTC 22:00 (JST 07:00)
- Push main (コード変更プッシュ)
- 手動 workflow_dispatch

ジョブフロー:

```
jobs:
  predict:
    steps:
      - Checkout
      - Setup Python 3.11
      - Install dependencies
      - Fetch temperature (Open-Meteo API)
      - Generate features
      - Train LightGBM
      - Train Ensemble
      - Train PyCaret
      - Predict Kanto
      - Predict Kansai-Hokuriku
      - Predict Kyushu-Okinawa
      - Predict Hokkaido
      - Merge predictions
      - Update data.js
      - Check anomalies (MAPE > 0.8 × 2σ)
      - Create GitHub Issue (if anomaly)
      - Copy artifacts to _site/
      - Upload Pages artifact
      - Deploy predictions to Pages
```

環境変数

```
env:
  PYTHONIOENCODING: "utf-8"  # 日本語対応
```

- .github/workflows/daily.yaml: 完全なワークフロー
- .github/workflows/fetch_temp.yaml: 気象データ取得
- .github/workflows/train_pycaret.yaml: PyCaret学習

Open-Meteo API仕様

エンドポイント

```
https://api.open-meteo.com/v1/forecast
```

パラメータ

パラメータ	値	説明
latitude	35.6785	東京緯度
longitude	139.6823	東京経度
hourly	temperature_2m	2m気温データ
timezone	Asia/Tokyo	JST(UTC+9)
past_days	7	過去7日間
forecast_days	7	未来7日間

レスポンス例

```
{
  "hourly": {
    "time": ["2025-01-07T00:00", "2025-01-07T01:00", ...],
    "temperature_2m": [5.2, 5.0, 4.8, ...]
  }
}
```

実装: 1.scripts/fetch_temp.py :

```
import requests

url = f"https://api.open-meteo.com/v1/forecast?latitude={LAT}&longitude={LON}&hourly=temperature_2m&timezone=Asia%2FTokyo&past_days=7&forecast_days=7"

response = requests.get(url, timeout=30)
data = response.json()
```

デプロイ仕様

GitHub Pages設定

項目	値
Source	GitHub Actions
Branch	必須無効
URL	https://13236584.github.io/Power-Demand-Forecast/

ディレクトリ構成

```
_site/  (GitHub Pages デプロイディレクトリ)
├── index.html
├── data.js
├── data/
│   ├── predict_log/
│   │   ├── latest.log
│   │   └── history/
│   │       ├── predict_20251231.log
│   │       ├── ...
│   │       ├── fetch_temp_20251231.log
│   │       ├── ...
│   │       ├── train_20251231.log
│   │       ├── ...
│   │       ├── train_pycaret_20251231.log
│   │       ├── ...
│   │       ├── backtest_20251231.log
│   │       ├── ...
│   │       ├── forecast_20251231.log
│   │       ├── ...
│   │       ├── ci_20251231.log
│   │       ├── ...
│   │       ├── anomaly
```